PROJECT SPECIFICATION / 01

Delegated Agent Control Plane

Project definition, architecture and reuse-first implementation blueprint

Design direction
Define the control-plane boundaries, evaluate OpenClaw Enterprise as the resource-management foundation, and build the product-specific task layer around reusable infrastructure. Connect user surfaces, agent runtimes, execution environments and application tools through explicit, versioned contracts.

The system lets an individual delegate software work, follow its progress, answer questions, approve specific operations and receive results with evidence. It preserves responsibility and execution history across interruptions and component changes.

Document control	Value
Version / date	1.1 / 12 October 2026
Status	Proposed build specification; no implementation is claimed
Audience	Product owners, architects, backend and platform engineers, runtime integrators, frontend engineers, QA and security reviewers
Working project name	Delegated Agent Control Plane (descriptive; branding remains open)
Initial product	A web workspace for delegated software-development tasks
Evidence labels	Requirement = intended behavior. Proposal = recommended design. Verified reference = externally documented capability. Open decision = needs implementation validation.

Version 1.1 change. Requirements remain explicit, but components are no longer assumed to be new builds. Evaluate reuse before implementation; sections 35-45 and the revised backlog govern component sourcing.

Completeness boundary. This document defines the proposed system and build gates. It is not Grok Bot source documentation, a claim of universal runtime compatibility, or a substitute for adapter prototypes, provider selection and security validation. Those unresolved decisions have explicit closure criteria.

Confidence. High in the responsibility boundaries and reliability requirements; moderate in the initial implementation choices; unknown in undocumented third-party internals.

PROJECT SPECIFICATION / 02

Reading guide and contents

Read sections 3-8 for the product and architectural boundaries; 9-17 for implementation contracts; 18-24 for trust, coordination and user experience; 25-31 for operations and delivery; 32-34 for examples and original references. Sections 35-45 add the reuse decision, integration analysis, evidence gaps and revised procurement/build plan.

Sections	Subject
03-05	Project charter; product scope; terms and identities
06-08	Architecture; ownership; component inventory
09-11	Domain model; storage and consistency; state machines
12-15	End-to-end flow; attention and approvals; events; public API
16-17	Runtime adapters; execution environment providers
18-20	Tool enforcement; identity and credentials; agent communication
21-24	Verification; failure recovery; user surface; memory and routines
25-27	Observability and cost; deployment; threat model
28-31	Acceptance tests; roadmap; build backlog; decisions and risks
32-33	Worked event and attention examples; contract checklist
34	Original architecture references
35-43	Reuse analysis; OpenClaw Enterprise; runtime/sandbox choices; revised boundaries and proof plan
44-45	Reuse research references and source evidence

How to use this specification

• Turn each work package in section 30 into implementation tickets. Keep its acceptance gate attached to the ticket.

• Use the entity definitions, states, event envelope and API semantics as the starting contract. Publish executable JSON Schemas and OpenAPI definitions during phase 0.

• Treat all numerical defaults as proposed configuration values, not benchmark results or market claims.

• Before integrating a runtime, record its exact version, license, supported transport, authentication method and capability test results.

• Revise architecture decisions through a recorded decision, rather than silently changing adapter behavior.

Source policy. Bracketed references such as [R2] point to the bibliography. Most of this document is original proposed design. Sources establish protocol or product capabilities only; they do not establish that this project already works.

PROJECT SPECIFICATION / 03

Project charter

Problem and intended outcome

Users can ask coding agents to do work, but a usable delegated-work product also needs durable ownership, execution tracking, environment management and a reliable return path when human input is required. This project supplies those responsibilities around agent runtimes without taking over their reasoning loops.

Primary user: an individual building or maintaining software who can describe an outcome and review evidence. Technical expertise may vary. The product must expose understandable task status rather than requiring users to interpret terminal logs.

Initial scenario: connect a repository, submit a bounded code change, run an agent in an isolated workspace, answer a question or approve an operation, review a diff and verification evidence, then choose the next action.

Objectives

- Preserve a task and its history independently of the chat surface and the live agent process.
- Support heterogeneous runtimes through tested contracts and declared capability limits.
- Route human input to the exact waiting task, attempt and native runtime request.
- Apply permissions outside model instructions wherever the platform owns the operation.
- Make cancellation, recovery, cost and verification visible and testable.

Product architecture decision

Separate responsibilities and contracts, while reusing existing implementations. Evaluate OpenClaw Enterprise/OCC for Agent deployment, infrastructure reconciliation, IAM and audit. Build a first-party task/attention/result product above that foundation only where required semantics are missing. Do not create a competing controller for the same runtime. A lighter runtime plus managed-sandbox composition is the fallback if OCC operations or fit are unacceptable. See sections 35-43.

Success evidence

The pilot succeeds when a real task completes with inspectable changes and evidence; a human wait survives a backend restart; a worker failure is reconciled without blindly repeating a side effect; and a second supported integration can be added without changing task semantics. Metrics and release tests are defined in sections 25 and 28.

Non-claim. Grok Bot is a behavior reference [R1-R4]. Public documentation is insufficient to reproduce its harness or establish its portability. This project does not depend on access to those internals.

PROJECT SPECIFICATION / 04

Product scope and packaging

Stage	Included behavior	Explicit boundary
First usable product	Web task workspace; repository connection; one runtime; isolated execution; questions and action approvals; durable status; diff/artifact review; cancellation and cost limits.	No promise of arbitrary runtimes or desktop automation.
Pilot extension	A second runtime or surface; structured delegation; agent profiles; scoped memory; routine scheduling; policy administration.	Add integrations only after capability and isolation tests.
Later products	Reusable API/SDK; embedded surfaces; local/private workers; enterprise controls; optional persistent browser workspace.	Each product has a separately approved scope and operating model.

Product surfaces over the same core

Personal workspace: the first end-user product. Embedded developer API: task and attention interfaces for applications, after contracts stabilize. Operations console: worker health, policy, audit and cost administration. These use the same task identities and policy checks; UI-specific data stays in the surface layer.

Commercial assumptions to validate

Evaluate a subscription with included usage and metered overage, or bring-your-own-provider credentials plus platform fees. No price is assumed. Determine model-provider resale terms, runtime licensing, account authentication restrictions and infrastructure costs before selecting billing. The prototype may use platform-funded test credentials; the pilot must make the chosen funding model explicit.

Out of scope for the first release

- Training a foundation model or building a full coding harness from scratch.
- Unrestricted autonomous publishing, production deployment or financial actions.
- General desktop control, automatic multi-agent swarms and cross-tenant agent messaging.
- Marketplace distribution, enterprise SSO and multi-region failover before core reliability is proven.

Definition of a usable release

A user can delegate, inspect progress, respond, stop work and review a result from the UI. Failures have actionable states. Unsupported runtime features are declared before execution. Production hosting is conditional on the security and recovery gates in section 28.

PROJECT SPECIFICATION / 05

Terms and identity boundaries

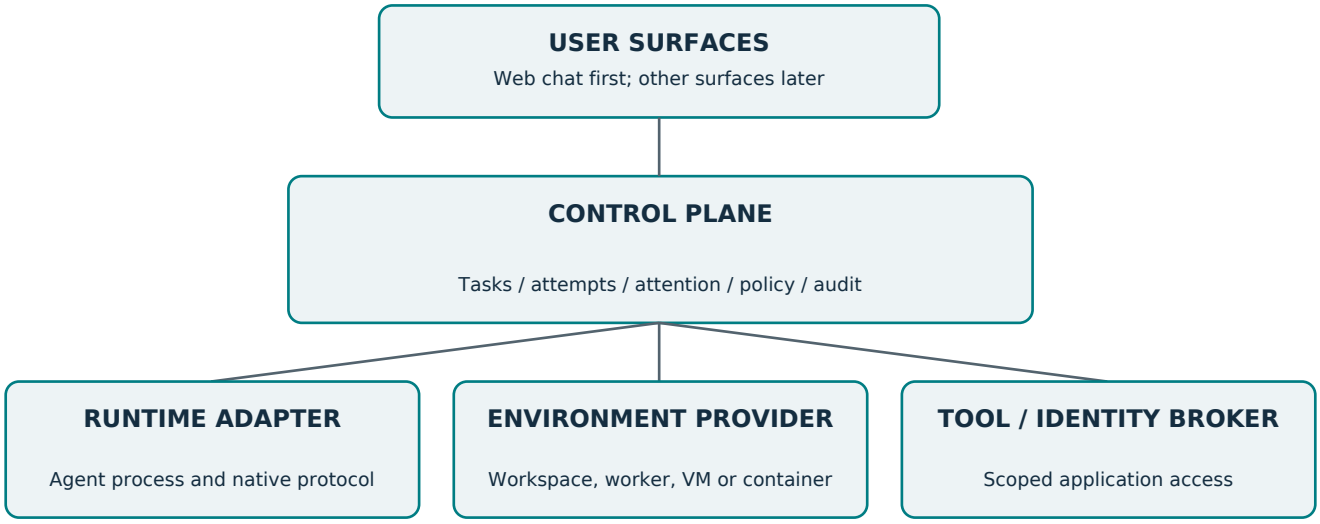
Term	Exact meaning
Tenant	Ownership and isolation domain. Initially one personal account; designed to support an organization later.
Principal	Authenticated human or service identity making a request. A model-generated message is not an authenticated principal.
Agent profile	Versioned role, instructions, tool policy references and memory scope. Not a machine or live process.
Task	Durable requested outcome, owner, acceptance criteria, budget and lifecycle.
Session	Runtime-specific continuing conversation/context associated with a profile and workspace. It may span multiple turns.
Turn	One submitted instruction and the runtime activity that follows it. End of a turn does not imply task acceptance.
Attempt	One execution effort for a task, with its own worker ownership, runtime binding and recovery history.
Environment	Provisioned compute and network boundary hosting work; may be a VM, container or private worker.
Workspace	Project filesystem and repository state. A durable workspace reference is distinct from a live environment.
Attention request	Durable question, action approval, authentication handoff or blocked-work request with a specific target.
Action intent	Description of a proposed operation, target, inputs and policy context used for enforcement.
Artifact / evidence	Persisted output / record supporting a claim about output correctness.
Control plane	Durable coordination and policy authority around execution.
Execution plane	Workers, runtime processes and tools performing operations.

Identity rules

Task IDs, session IDs, attempt IDs and environment IDs are separate opaque identifiers. Native runtime IDs are adapter mappings, never public authorization tokens. Every persisted object carries tenant ownership. Profile identity does not create an external account: external authority comes from a separately granted connection or workload identity.

PROJECT SPECIFICATION / 06

System architecture



Solid links are interface boundaries, not a prescribed service deployment.

Three interface families

Surface contract: submit tasks, receive events, answer attention requests, inspect artifacts. Runtime contract: create/bind sessions, submit turns, normalize events, handle decisions and cancel. Environment contract: provision compute, prepare workspace, launch/stop processes and collect outputs. Tool and identity brokers enforce application access alongside these interfaces.

Control-plane responsibilities

The composed control plane owns task coordination, execution records, attention lifecycle, policy decisions, budgets and audit. Reuse OCC resource lifecycle/IAM where adopted; the product owns outcome-oriented task semantics. Persist state independently of live sockets. Schedule work with durable dispatch. Reconcile execution outcomes after disconnects. Treat model output as an input to coordination, not privileged authority.

Execution-plane responsibilities

Run agent processes and commands, access permitted model endpoints, read/write the workspace, and emit bounded progress events. The model chooses actions through its runtime; the control plane does not need to recreate its reasoning loop.

Deployment interpretation

The diagram defines boundaries, not mandatory microservices. The product backend can package task, attention and result modules together, while OCC retains its own resource-management ownership when adopted. Run untrusted execution workers separately. The runtime adapter can be a worker-side library; the environment provider may be a managed service. Maintain versioned interfaces so components can be replaced.

Isolation proposal. Begin with an isolated environment per active task, or per explicit trust group after review. Do not copy Grok Bot's shared per-user computer policy by default. Shared machines trade easier handoff for shared credential and filesystem exposure [R2].

PROJECT SPECIFICATION / 07

Who owns each responsibility?

These are logical owners, not a list of new services to write. Map each to the adopted foundation, runtime or product layer using sections 8 and 35-43.

Responsibility	Owner	Boundary / invariant
Requested outcome and acceptance	Task service	Agent turn completion cannot independently accept the task.
Model calls and reasoning context	Runtime	Adapter reports behavior; control plane does not inspect hidden reasoning.
Session persistence	Runtime + adapter	Control plane stores mapping and capability; native checkpoints remain runtime-specific.
Work dispatch and retries	Dispatcher	Retries create auditable attempts; uncertain effects require reconciliation.
Machine and process lifecycle	Environment provider + worker	No environment lifecycle request bypasses tenant policy.
Human questions / decisions	Attention service	Each answer targets a current request; approval is bound to an action.
Application authentication	Identity / connector broker	Model prompts never become credential storage.
External action permissions	Policy + executing broker	Enforce on the execution path; profile instructions alone are insufficient.
Agent-to-agent handoffs	Coordination service	Explicit ownership, deadlines, context references and capability limits.
Code acceptance and evidence	Verification service + reviewer	Runtime self-report is a claim, not the only completion criterion.
User presentation	Surface	Backend remains authority; notifications are not durable task state.
Resource accounting	Usage service	Record model, compute and storage usage per attempt where measurable.

Split control from task judgment

The control plane decides whether work may run, who owns it and how state changes. An optional supervising agent decides how to break down a task or review output. A supervisor uses the same APIs and cannot grant permissions, bypass budgets or accept another user's approval.

PROJECT SPECIFICATION / 08

Components: reuse, integrate and build

Component	Sourcing decision / remaining responsibility
Agent deployment / controller	EVALUATE REUSE: OCC resources, immutable revisions, durable work and Compute Drivers. Do not rebuild these before testing fit [R11-R13].
IAM and infrastructure audit	REUSE where adopted: OCC exact-resource authorization and audit; extend product task permissions at the public boundary, without granting broad operator authority.
Agent reasoning/runtime	REUSE: OpenClaw runtime and supported Codex bridge, or another tested structured runtime. Build only normalization and product correlation.
Task lifecycle / public API	BUILD: objective, criteria, task/attempt states, budgets, result review and surface-independent identities. OCC deployment state is not this task model.
Attention / user decisions	INTEGRATE native prompts; BUILD durable product inbox/correlation. Preserve native expiry and execution authority; do not duplicate authorization.
Execution environment	REUSE one supported Compute/provider stack. Firecracker is a low-level option; E2B/OpenSandbox/Daytona provide higher-level lifecycle components [R18-R21].
Credentials / connectors	REUSE secret stores and native supported bindings. BUILD only task-scoped grants or missing application connections. OpenShell pairing remains experimental [R15-R17].
Artifacts / verification	REUSE native diffs, logs and existing CI; BUILD immutable result manifests, criterion mapping and user acceptance.
Chat and channels	REUSE Gateway API and existing channel plugins where appropriate; BUILD first-party task views and authenticated identity mapping.
Usage / routines / coordination	REUSE native telemetry, schedules and subagent mechanics if qualified; BUILD cross-runtime task budgets, ownership and evidence mappings.

Meaning of build: implement only missing product semantics or a thin adapter. Every implementation ticket must identify its upstream capability, gap and owning boundary. The earlier requirement inventory is not permission to rebuild entire subsystems.

PROJECT SPECIFICATION / 09

Domain model and relationships

Entity	Minimum fields and relationships
Tenant / principal	id, membership, roles, policy_version; principals may belong to authorized tenants.
Project / workspace	id, tenant_id, repository_connection_id, base_revision, storage_ref, trust_group.
AgentProfileVersion	profile_id, version, instructions_ref, runtime_constraints, tool_policy_ref, memory_scope.
Task	id, tenant_id, project_id, owner_id, profile_version, objective, acceptance_criteria, state, version, budget_ref.
Attempt	id, task_id, runtime_adapter_version, session_binding_id, environment_id, state, lease_epoch, started_at, ended_at.
SessionBinding	id, tenant_id, native_session_ref, runtime_version, workspace_id, checkpoint_ref, recovery_mode.
AttentionRequest	id, task_id, attempt_id, type, native_request_ref, action_hash, payload_ref, status, expiry, authorized_responders.
Decision / delivery	request_id, principal_id, decision, decided_at, request_version, dispatch_id, delivered_at, consumed_at.
Action / operation	id, attempt_id, target, normalized_input_hash, policy_version, grant_ref, execution_status, external_operation_ref.
Connection / grant	connection_id, owner_principal, provider, credential_ref, scopes; grants bind permitted use to a task/profile.
Artifact / verification	artifact_id, task_id, attempt_id, URI, digest, classification, source_revision; verification_id and result.
Event / handoff	event_id, aggregate_id, sequence; handoff_id, parent_task_id, child_task_id, owner, deadline, context_refs.

Cardinalities: one task has many attempts and attention requests; an attempt binds one primary runtime session and environment; a workspace may be reused through serialized ownership; child tasks reference a parent. A native session can host multiple sequential turns, but concurrent turns require a declared runtime capability. Separate task sessions by default to prevent context leakage.

PROJECT SPECIFICATION / 10

Storage and consistency

Proposed initial storage

For product task state, use PostgreSQL for attempts, decisions, connection metadata, dispatch commands and event metadata. If OCC is adopted, it remains authoritative for its Agent resources, revisions and controller work. Keep product tables in a separate owned schema/database and integrate through APIs; never write OCC private tables. Object storage holds artifacts, large logs and encrypted checkpoints. Store secret material in a managed secret store, with opaque references in the database. An event stream is a delivery mechanism; it is not the sole state store.

Atomic state and dispatch

In one database transaction: validate tenant and expected object version; apply the state transition; increment the aggregate sequence; insert audit/event records; insert dispatch commands into an outbox. A publisher delivers outbox commands at least once. Consumers deduplicate by command/event ID. A crash after commit must not lose dispatch; a crash after delivery must not duplicate the logical transition.

Worker leases and fencing

Product workers claim task work using compare-and-set and receive an incrementing lease epoch. Heartbeats extend a bounded lease. Persisted writes and broker actions must include the current epoch; stale owners are rejected. Provider reconciliation must stop or quarantine an old environment before giving a replacement worker conflicting authority. Database fencing alone cannot undo external actions already started.

Database constraints

- Tenant-scoped foreign keys or equivalent validated relationships; deny cross-tenant joins and object retrieval.
- Unique idempotency key per tenant and API operation; unique native event key per adapter binding.
- At most one accepted decision per attention request version; no decision after expiry or supersession.
- At most one active attempt per task in v1; multiple child tasks provide controlled parallelism later.
- Append-only security audit; artifact digests and finalized manifests cannot be silently overwritten.

Ordering and replay

Use a monotonically increasing sequence per task aggregate. Worker events include a local sequence; ingestion buffers or reconciles gaps before applying dependent transitions. There is no global order guarantee. UI clients reconnect with a cursor and replay authorized events, then fetch the current snapshot if the cursor expired.

Reliability rule. At-least-once delivery plus deduplication does not imply exactly-once external effects. Non-idempotent operations need an external idempotency key or a reconciliation procedure.

PROJECT SPECIFICATION / 11

Task and attempt state machines

Task state	Meaning / permitted next states
draft	Editable intent; submit -> queued.
queued	Accepted for execution; dispatch -> running; cancel -> cancelled.
running	Active work; wait -> waiting_attention; output -> verifying; error -> failed; stop -> cancelling.
waiting_attention	A required answer or authorization blocks work; consumed response -> running; timeout/denial -> running with restriction, failed or cancelling according to request policy.
verifying	Evidence and acceptance checks execute; pass -> ready_for_review or succeeded if explicit auto-accept policy; mismatch -> running/failed.
ready_for_review	Result available; authorized accept -> succeeded; request_changes -> queued new attempt; cancel -> cancelled.
cancelling	Stop requested, not yet confirmed; reconciled stop -> cancelled; uncertainty -> remains cancelling with escalation.
succeeded / failed / cancelled	Terminal task outcome. New work or an explicit retry creates a recorded new attempt; cancellation does not undo effects.

Attempt states: allocated -> provisioning -> starting -> executing -> waiting -> executing -> finalizing -> completed. Failure paths: failed, lost, cancellation_requested and terminated. Task state summarizes user-visible ownership; attempt state records execution lifecycle. A lost attempt does not automatically mean a failed task.

Transition rules

• Require authenticated command, expected version and a permitted transition. Terminal events from stale attempts are retained for audit but cannot override the current owner.

• A native end-of-turn moves work to verification only when the adapter reports a proposed result. Idle, token limit and user-cancel stop reasons are not successful task completion.

• One blocking attention request per attempt in v1. Additional suggestions may be nonblocking events. Later parallel branches need independent child-task state.

• Denial blocks the proposed action; it need not cancel the entire task. A runtime that cannot continue after denial must report that limitation.

Environment states

requested -> provisioning -> ready -> active -> draining -> stopped -> destroyed; failed/quarantined are exceptional states. Workspace retention is separate from compute termination. Cleanup is not complete until the provider confirms termination and credential grants are revoked.

PROJECT SPECIFICATION / 12

End-to-end execution workflow

Step	System behavior	Durable record
1. Submit	Authenticate user; authorize project; validate criteria, capability and budget.	Task + submitted event + dispatch command.
2. Prepare	Claim attempt; provision isolated environment; fetch pinned repository revision.	Attempt, lease, environment and workspace revision.
3. Start	Launch pinned runtime; bind native session; provide scoped tool access and instructions.	Session binding, adapter version, granted capabilities.
4. Execute	Runtime reasons and acts; worker normalizes progress; brokers enforce owned actions.	Progress, tool outcomes, operation IDs and usage.
5. Ask / approve	Persist attention request before displaying it; suspend only affected work.	Request, proposed action and waiting state.
6. Respond	Authenticate responder; commit decision; dispatch to matching native request.	Decision, delivery and consumption acknowledgment.
7. Verify	Collect diff and evidence; run acceptance checks against exact result revision.	Artifact manifest and verification records.
8. Review	Show result, remaining limitations and external effects; accept or request changes.	Review decision and terminal/new attempt state.
9. Clean up	Revoke short-lived grants; stop compute; retain outputs under retention policy.	Provider termination and cleanup outcomes.

Working example

A user asks to fix a login error and adds an acceptance criterion: the existing regression suite passes and the failing path has a reproducible test. The agent receives the repository and tools needed to edit and run tests. It asks which of two intended behaviors is correct. The user answers from the attention card. The same native request receives that answer, or a replacement attempt receives the recorded answer if recovery cannot restore the session. The final review shows the patch, exact test commands, exit codes and revision.

External publication is a separate action. Creating a draft PR may be permitted by task policy; merging or deploying requires its own grant or approval. The UI must not equate local code changes with a shipped product.

PROJECT SPECIFICATION / 13

Human attention and approval protocol

Attention types

Question: missing task information. Action approval: authorization for a specific operation. Authentication handoff: secure login/OAuth flow outside chat. Blocked work: an environment, permission or resource problem requiring a choice. Each type has a different response schema.

Durable request lifecycle

created -> pending -> answered -> dispatching -> delivered -> consumed -> closed. Alternative terminal states: denied, expired, cancelled and superseded. An answer is not proof that the runtime resumed. Retry delivery with the same decision ID; show a delivery problem if consumption cannot be established.

Approval binding

Bind approval to tenant, authorized principal, task, attempt, native request, operation, target, canonical input hash, policy version and expiry. Display a human-readable summary plus the exact affected target and consequential inputs. Changed inputs, target or policy require reevaluation; changed action content requires a new approval. Revalidate authority immediately before execution.

Concurrent answers and stale screens

Use an expected request version and atomic compare-and-set. First valid decision wins; later submissions return a conflict with the recorded decision. Expired or superseded cards cannot authorize a new attempt. A notification link only opens the authenticated request; clicking the notification itself never approves work.

Restart and cancellation behavior

If the native process remains available, redeliver to its mapped request. If it died, mark the original request superseded and reconcile the proposed action. A new attempt may reuse a non-sensitive factual answer as task context, but must create a fresh authorization when its action or identity binding differs. Cancelling an attempt invalidates pending authorizations and native waits.

Adapter requirement. An approval UI is trustworthy only when the actual execution path waits for and enforces the decision. A transcript parser that notices an action after execution cannot implement this boundary. ACP provides permission request mechanisms, but support must be validated per runtime/version [R5].

PROJECT SPECIFICATION / 14

Event and command contracts

Event envelope v1

```
{
  "schema_version": "1.0", "event_id": "evt_opaque",
  "tenant_id": "ten_opaque", "task_id": "task_opaque",
  "attempt_id": "att_opaque", "sequence": 17,
  "type": "attention.requested", "occurred_at": "RFC3339",
  "received_at": "RFC3339", "producer": "adapter/runtime@version",
  "correlation_id": "corr_opaque", "causation_id": "evt_or_cmd",
  "payload_ref": "authorized-object-reference",
  "classification": "project_internal"
}
```

The authenticated ingestion boundary supplies or verifies tenant and attempt identity; it does not trust these fields merely because the worker sent them. Keep large/sensitive content in an access-controlled object, not an unrestricted event bus. Event IDs deduplicate delivery; sequence orders the task projection.

Event family	Examples
Task / execution	task.submitted, attempt.allocated, environment.ready, runtime.started, attempt.heartbeat, attempt.lost, task.cancel_requested
Runtime progress	turn.started, message.delta, tool.proposed, tool.finished, turn.finished, checkpoint.created
Attention / policy	attention.requested, decision.recorded, decision.delivered, decision.consumed, action.denied
Results / accounting	artifact.created, verification.finished, result.accepted, usage.recorded, budget.threshold, cleanup.finished

Commands versus observations

Commands request an action: StartAttempt, SubmitTurn, DeliverDecision, CancelAttempt, VerifyResult and CleanupEnvironment. Events report an observation. Each command carries command_id, tenant_id, target_id, expected_version where relevant, lease_epoch, deadline and idempotency_key. An observation cannot grant permission simply by naming itself action.approved.

Compatibility and data limits

Additive fields are backward compatible within v1. Breaking changes require a major contract version. Unknown optional progress events are stored and displayed generically; unknown permission or state-changing semantics are rejected. Proposed default: 256 KB maximum inline event payload; larger content is uploaded separately. Validate and redact before fan-out.

PROJECT SPECIFICATION / 15

Public API and surface contract

Endpoint / method	Semantics
POST /v1/tasks	Create and submit objective, criteria, profile, project, runtime constraints and budget. Require Idempotency-Key.
GET /v1/tasks/{id}	Return current snapshot, version, active attempt, attention summary and result references.
GET /v1/tasks/{id}/events	SSE/replay with cursor; authorize task read; preserve per-task sequence.
POST /v1/tasks/{id}/messages	Add instruction with message ID; adapter declares queue vs interrupt semantics.
POST /v1/tasks/{id}/cancel	Request cancellation; return 202 until execution stop is reconciled.
POST /v1/tasks/{id}/retry	Start a new attempt after explicit recovery checks; never silently repeat uncertain effects.
GET /v1/attention	List current user-actionable requests scoped to tenant and responder authority.
POST /v1/attention/{id}/responses	Validate typed answer/decision and expected version; persist before dispatch.
POST /v1/tasks/{id}/reviews	Accept result or request changes; require result version and reviewer authority.
GET /v1/artifacts/{id}	Authorize download/view; issue short-lived access; no public object-store URLs.
POST /v1/connections / .../revoke	Start secure connection flow / revoke authority and propagate to live tasks.
GET /v1/capabilities	Expose supported runtimes, environments and tested feature constraints.

Response and error rules

Return 202 with a durable operation ID for async work. Use 401 unauthenticated, 403 unauthorized, 404 absent or hidden resource, 409 stale version/state, 422 unsupported capability or invalid input and 429 quota. Include machine-readable error_code, correlation_id and actionable message. Do not leak other tenants' resource existence.

Webhooks and embedded surfaces

Later webhooks are signed, timestamped, retried and delivered at least once with event IDs. Verify replay windows and rotate secrets. A Slack/mobile adapter maps external identity to an authorized principal; possession of a channel or webhook URL is not authority to approve. The authenticated backend remains the decision authority.

PROJECT SPECIFICATION / 16

Runtime adapter contract

Operation	Required behavior
discover_capabilities()	Return pinned runtime/protocol versions and tested support for events, permissions, questions, cancellation, continuation and checkpoints.
create_or_bind_session(context)	Return native session binding; declare workspace and concurrency requirements; isolate native metadata.
submit_turn(binding, instruction)	Deduplicate command; report accepted, rejected or uncertain. Do not infer acceptance solely from process spawn.
stream_or_poll_events(binding)	Normalize native messages, tool status, stop reasons, usage and attention; preserve source IDs.
deliver_response(request, decision)	Map to exact native request; acknowledge delivery and consumption when observable.
cancel(binding, reason)	Request stop; report confirmed, pending or unsupported; escalate to environment termination if necessary.
recover(binding, checkpoint)	Declare exact resume, restart-with-context or unsupported; never label replayed work exact resume.
collect_result(binding)	Return outputs and claims with revision/workspace identifiers; no automatic task acceptance.

Capability tiers

Tier A: structured events, enforceable permissions, cancellation and recoverable native sessions. Tier B: structured events and permissions, with restart-from-artifacts recovery. Tier C: headless output without adequate control callbacks; restricted to low-authority tasks. A Tier C adapter must not advertise action-level approvals.

Protocol and runtime selection

Reuse the OpenClaw Gateway/Codex bridge where fit is demonstrated; otherwise prefer a native structured API or negotiated ACP version [R5]. Do not rewrite an existing supported harness. OCC currently documents embedded OpenClaw and dedicated Codex topologies [R12]. Pin protocol schemas: ACP versions differ and resume is capability-dependent. Grok Build documents headless and ACP use [R10], making it a candidate, subject to tests and account/licensing review. Codex, Claude Code and other runtimes remain candidate integrations until their current official interfaces are separately validated. No interchangeability is assumed.

Conformance gate

A simulator first exercises permissions, disconnects and duplicate events. This is a test fixture, not a replacement agent harness. Then a real runtime must pass create, follow-up, permission, denial, cancel, crash and artifact tests. If its shell can bypass the gateway, restrict network and credentials or reject high-authority workloads. Native approval semantics must be preserved, not translated into broader grants.

PROJECT SPECIFICATION / 17

Execution environments and workers

Environment provider interface

Define a normalization layer over the selected provider; do not implement fleet orchestration already owned by OCC or a sandbox platform. The proposed interface includes provision(spec), inspect(id), prepare_workspace(id, revision), launch_process(id, spec), terminate_process(id), export_artifacts(id), snapshot_workspace(id), stop(id) and destroy(id). Every asynchronous operation returns an operation ID and has a reconciliation method. Credentials and network policy are inputs by reference, never literal fields in general logs.

Environment option	Use and constraints
Isolated managed VM / sandbox	Recommended pilot candidate for untrusted repository execution; validate provider isolation, network controls, lifecycle and persistent storage.
Local development container	Suitable for controlled development fixtures. Not the default hostile multi-tenant boundary.
Self-managed microVM	Later option if hosting requires it. Firecracker supplies a VMM, not the full worker fleet, storage or lifecycle platform [R9].
Private / local worker	Later opt-in for special OS/tooling or private-network access; enrolled with revocable workload identity and explicit machine authority.

Workspace and parallelism

Pin a repository base revision and create a dedicated working branch/worktree. Never share a writable checkout between unrelated attempts. Handoffs use immutable artifacts or separately authorized workspace access. A shared project can support parallel child tasks in distinct worktrees; integration occurs through a merge/review step with conflict handling.

Machine controls

Apply CPU, memory, disk, duration and egress budgets. Run without host credentials, privileged mounts or container-engine sockets. Register workers through short-lived enrollment and scoped workload credentials. Proposed defaults: heartbeat every 15 seconds, suspect after 60 seconds, lease expiry after 90 seconds; tune through fault tests. Expiry triggers reconciliation, not automatic blind replay.

Browser computer use

Optional later module: isolated browser profile and desktop session, controlled input, authenticated viewer, secure takeover and an explicit control lease. Human takeover pauses conflicting automation. Persist browser sessions only under a defined trust scope. This is separate engineering work, not a free consequence of provisioning a VM.

PROJECT SPECIFICATION / 18

Tool gateway and policy enforcement

Deterministic execution boundary

For platform-mediated operations, runtime proposes an action intent; policy evaluates identity, grant, target, operation, inputs, classification and budget; the gateway permits execution or creates an attention request. The executing component verifies a short-lived authorization token bound to that intent. Record the operation ID before dispatch and reconcile its outcome afterwards.

Action class	Proposed default
Workspace reads / edits	Allow within the assigned workspace and resource limits.
Tests / builds	Allow in the sandbox with approved network access and no privileged mounts.
Connected application reads	Allow only with scoped connection grant; redact restricted outputs.
External writes / publication	Ask unless an explicit bounded policy authorizes the operation.
Merge / production deployment	Ask with target, revision, checks and effect shown.
New credentials / broader access	Secure auth flow plus explicit grant; model cannot approve itself.
Destructive or unknown operation	Deny or require clarified intent; never infer broad authority from vague approval.

Shell and browser bypass problem

A runtime with arbitrary shell access, unrestricted egress and valid credentials can act outside a connector gateway. Therefore connector policy is not sufficient. Use narrow credentials, network restrictions, sandbox isolation and runtime permission enforcement. If a workload cannot be controlled at the actual action path, do not claim per-action approval guarantees; reduce its authority or exclude it from that release.

AI review role

An optional review model can classify ambiguous actions or suggest risk explanations. It cannot grant authority beyond deterministic policy. Treat uncertainty as ask/deny according to task policy. Keep reviewer input/output auditable; do not rely on the agent's own narrative as the only action description.

Policy changes

Version policies and revalidate before dispatch. Revocation takes priority over cached grants. Separate a resource-access rule from a human-approval rule: approving an action does not grant an absent connection or override tenant restrictions. MCP is an integration protocol; the platform still owns its grant and execution policy [R6].

PROJECT SPECIFICATION / 19

Identity, credentials and connections

Reuse the selected platform/runtime authentication and secret mechanisms first. The table defines required ownership; task-specific grants and unsupported app connections are the remaining extension points.

Layer	Required ownership and behavior
Platform identity	Authenticate people via an identity provider; tenant membership and roles checked server-side. Start with managed sign-in; organization SSO later.
Workload identity	Workers/adapters authenticate as scoped services. Bind credentials to tenant, attempt, lease epoch and expiry; rotate and revoke.
Connector credentials	Broker stores OAuth/API secrets encrypted. Runtime gets a connection reference, not the raw credential, when mediation is possible.
Browser identity	Browser sessions belong to an explicit user/trust scope. Site-side permissions govern actions; isolate profiles from unrelated work.
CLI credentials	Short-lived restricted credentials injected only into necessary processes. Never bake them into images or shared artifact directories.
Model-provider credentials	Separate funding/auth method from app credentials. Per-tenant BYOK or platform-funded credentials require distinct budgets and terms review.

Connection lifecycle

requested -> authorizing -> active -> refresh_needed -> active; revoked/expired/error are blocked states. Store provider, external account identifier, granted scopes, owner principal, credential reference and authorized use grants. A profile does not automatically inherit every user connection. Grant read/write capabilities deliberately for a task or explicit reusable role.

Secure authorization

Use provider-supported authorization-code flows, PKCE where applicable, validated callback state, exact redirect handling and audience-bound tokens. Follow RFC 9700 [R8]. Pin the MCP authorization version for HTTP integrations; STDIO servers use a different credential path [R6]. OAuth authorizes access but does not express the entire task policy.

Revocation and secret entry

A secure form or OAuth redirect handles secrets outside chat. Revocation blocks new calls, invalidates cached grants and interrupts or quarantines tasks that require the connection. An in-flight external action may already have occurred; reconcile it. Scrub secrets from logs, command traces, artifacts and support exports. Deleting a Bot/profile must not be mistaken for revoking external credentials.

PROJECT SPECIFICATION / 20

Agent coordination and communication

Profiles, supervisors and workers

A supervisor is an agent profile tasked with coordination, not a privileged platform administrator. Reuse native OpenClaw/Codex delegation where supported; add durable product ownership and evidence mappings rather than a competing subagent engine. It can propose child tasks, request specialist review and summarize progress. The coordination service validates every delegation and preserves parent-child relationships. Agents may execute on different machines; co-location is not required for communication.

Handoff field	Required meaning
Identity	handoff_id, sender_profile, recipient_profile, parent_task, child_task and tenant.
Work	Objective, acceptance criteria, scope, deadline and owner of the next step.
Context	Authorized artifact references, selected summary, source revision and classification.
Authority	Tool-grant references and resource budget; recipient permissions cannot exceed approved delegation.
Response	Accepted/rejected; progress; result reference; blocker; outcome and evidence.

Allowed communication

Permit task-scoped summaries, questions, evidence and authorized artifact references. Deny raw secret transmission, cross-tenant messages and unauthorized workspace references. Filter data by sender and recipient grants. A handoff transfers selected context, not automatic access to the sender's full conversation or memory. Audit who shared which classified references.

Coordination constraints

One owner per task; explicit dependency edges; detect cycles; limit depth, child count, concurrency and aggregate budget. Proposed pilot defaults: depth 2, at most 4 active child tasks, no recursive self-spawning. Child cancellation propagates from parent cancellation unless a recorded detached-work policy exists. A child result becomes parent input; it does not independently accept the parent outcome.

Delivery semantics

Messages have durable IDs and at-least-once delivery with deduplication. A recipient acknowledges acceptance; timeout creates a blocker. Shared files are not a substitute for a handoff record. Preserve provenance when results are summarized. Treat received agent text as task data, not as authority to change permissions or credential scope.

PROJECT SPECIFICATION / 21

Results, evidence and acceptance

Define completion before execution

Task creation requires an objective and acceptance criteria. For code changes, capture the repository base revision, requested behavior, allowed change scope and necessary checks. The system can propose criteria, but the task owner must see them before work begins.

Evidence item	Minimum record
Code change	Base and result revision, diff/patch reference, changed-file list and digest.
Test / build	Exact command, environment/image version, result revision, start/end, exit code and log reference.
Visual proof	Screenshot/output reference and capture context; required viewport or scenario where relevant.
External action	Operation ID, affected target, external receipt/reference and reconciled status.
Agent claim	Summary of work, limitations, checks not run and unresolved concerns.
Review	Verifier identity/version, criteria results and authorized acceptance or change request.

Verification pipeline

Collect a finalized artifact manifest; execute required checks in an appropriately isolated verifier; compare results to criteria; present evidence to the user. A failing check returns work for revision or marks a failure. Automatic acceptance is available only under an explicit task policy with objective checks. Otherwise the result enters ready_for_review.

Evidence integrity

Bind checks to the exact output revision. Results from an earlier patch do not validate later edits. Preserve immutable evidence and timestamps. A malicious repository can manipulate test output, so label agent-run evidence separately from platform-verifier evidence and run untrusted code under the same isolation requirements. Signed manifests establish provenance, not correctness.

Deliverable versus publication

A generated patch, draft PR or deploy preview is a deliverable. Merge, release and production deployment are independent actions with their own authorization. Show which actions actually happened and which remain proposed. Do not mark task success solely because a runtime said "done" or its process exited with code zero.

PROJECT SPECIFICATION / 22

Failure recovery and cancellation

Failure	Required handling
UI / socket disconnect	Work continues if authorized; reconnect retrieves snapshot and replays events. Do not start a replacement attempt.
Backend restart	Replay committed outbox commands and pending timers; preserve decisions and state.
Worker heartbeat loss	Mark suspect; inspect provider/runtime; fence stale owner; reconcile before retry.
Runtime process crash	Recover native session only if supported; otherwise preserve artifacts and start a new recorded attempt with selected context.
External write timeout	Mark operation outcome uncertain; query external system or require user reconciliation. Never blindly repeat.
OAuth expiry / revocation	Block affected calls; secure reauth or revised scope; never copy secrets into chat.
Duplicate decision / event	Deduplicate; reject version conflict; do not execute twice.
Disk / compute exhaustion	Stop or fail with artifact preservation where possible; cleanup provider resources.
Budget exhausted	Stop dispatching further work; terminate according to policy; report possible in-flight charges/effects.
Cancellation ignored	Escalate from native cancel to process kill/provider stop; remain cancelling until confirmed or explicitly uncertain.

Checkpoint versus replay

A checkpoint can contain native session state, workspace revision and pending-request mappings. Treat checkpoints as sensitive tenant data. A replacement environment must validate image/runtime compatibility. Resubmitting old prompts is a new execution attempt; it can repeat operations and is not exact recovery.

Retry policy

Retry transient provisioning and idempotent broker reads with bounded backoff and jitter. Never retry user denial, invalid credentials, unsupported capability or policy rejection as if transient. Proposed default: up to 3 automatic attempts for safe operations, then actionable failure. A task retry records its reason, previous effects and chosen recovery mode.

Cancellation contract: cancellation stops future work; it does not undo past effects. Persist cancel_requested before dispatch, invalidate pending authorizations, propagate to children and revoke unused grants. Report confirmed cleanup separately from the user's request to stop.

PROJECT SPECIFICATION / 23

First-product user experience

Screen / interaction	Information and controls
Task creation	Objective, project/repository, acceptance criteria, supported runtime/environment, budget and allowed external effects.
Task workspace	Status, current owner/attempt, concise activity, changed files, elapsed time, usage and attention summary.
Attention inbox	Reason, proposed effect or missing information, exact target, deadline and typed response controls.
Action approval	Current/proposed values or command; permitted scope; approve once/deny; clearly separate persistent rules.
Results review	Diff/artifacts, verification evidence, unresolved limitations, actions already performed, accept/request changes.
Connections	Connected account and scopes; grant/revoke controls; secure sign-in without chat credentials.
Execution controls	Follow-up, stop, safe retry; explain unsupported continuation or recovery capabilities.
Operations view	Worker status, uncertain effects, cleanup failures and auditable intervention controls.

Interaction requirements

Use plain-language status derived from backend state. Distinguish waiting for you, waiting for infrastructure, verifying and ready for review. Show a pending decision delivery separately from resumed work. Preserve keyboard access, readable contrast, accessible form labels and mobile-width layouts. Stream progress with bounded batching; the transcript is not the only navigation method.

Approvals and drafts

Put consequences before technical logs. For a publish operation, show destination and revision. For a command, show scope and why access is needed. Request scope changes explicitly. A persistent approval rule must have its own reviewable configuration and cannot be implied by a general "continue" response.

Later surface adapters

Slack, mobile and API surfaces may initiate or inspect tasks. Approval requires verified identity mapping and response correlation. Route sensitive authentication to a secure page. All surfaces use the same task and request IDs, so switching devices does not create new execution state.

PROJECT SPECIFICATION / 24

Memory, routines and configuration

Memory scopes

Keep task context, profile memory, project knowledge and tenant policy separate. A profile may retain stable preferences and summaries under explicit scope. Store provenance, last_verified_at, classification and expiry. Changing facts should be retrieved from their source. Do not make all profiles share all memory by default.

Configuration versioning

Version profile instructions, tool permissions, runtime configuration, environment images and policy. Reuse native memory and scheduling when their scope and durability satisfy these requirements; the product owns task-template/budget mappings, not a second native scheduler. Each attempt records the versions used. Applying a new profile version to active work is an explicit operation; security revocations apply immediately. Reproduction requires the versioned inputs, not only a role name.

Routine scheduling

A routine defines owner, task template, timezone, schedule/event trigger, authorization policy, concurrency rule, budget and expiry. Every firing creates an ordinary task with a trigger ID. Deduplicate firings; use no-overlap by default; record missed runs and choose skip or catch-up explicitly. Scheduled work does not inherit unlimited future authorization.

Routine concern	Required behavior
Time and daylight changes	Store IANA timezone; define wall-clock scheduling and ambiguous/missing-time handling.
Stale connections	Pause routine and request reauthentication; do not retry forever.
Changed source or permissions	Reevaluate grants/policy each firing; ask if authorization no longer covers the operation.
Deletion / disable	Stop new firings; separately choose whether active tasks are cancelled.
Recurring failures	Bound retries, notify owner and pause after configured threshold.

Shared templates

Later profile/routine templates export only selected configuration. Exclude secrets, browser sessions, private memory and live schedules. Import creates new identities and requires new connections. A published template is untrusted configuration; validate tools and requested authority before activation.

PROJECT SPECIFICATION / 25

Observability, budgets and service targets

Telemetry model

Attach tenant-safe task_id, attempt_id, command_id and correlation_id to operational telemetry. Record state transitions, dispatch delay, adapter errors, attention delivery, policy outcomes, resource usage and cleanup. Do not export secrets or full conversation bodies by default. Audit access and operational logs have separate retention and access policies.

Metric / proposed pilot target	Meaning
Task acknowledgment: p95 under 2 s	API persistence acknowledgment, excluding provisioning/model work.
Warm execution start: p95 under 30 s	Only for a documented warm provider path; measure cold start separately.
Decision delivery: p95 under 5 s	Committed answer to adapter acknowledgment when runtime is connected.
Replay completeness: 100% in tests	All committed critical state events recovered across tested restarts.
Cleanup: 99% within 5 min	After confirmed stop, excluding provider outages; alert on orphan resources.
Availability: 99.5% pilot objective	Core task/attention API; establish measurement window before commitment.

These are proposed engineering objectives, not measured performance or a contractual SLA. Model latency and external-provider uptime must be reported separately. Establish baselines before promising user-facing latency.

Cost model and enforcement

Task cost = model usage + compute duration + storage/egress + connector charges + platform overhead. Track known usage per attempt and reconcile delayed provider billing. Reserve estimated budget before dispatch, release unused reservation, and halt further work at the configured ceiling. In-flight usage can overshoot; disclose the enforcement limit and cap concurrency to bound it.

Operations and retention defaults

Proposed pilot defaults: verbose execution logs 7 days; task artifacts/history 30 days; security audit metadata 90 days. Provide configurable deletion and explicit backup expiry. Establish daily database backups, restore drills and cleanup sweeps. Alerts cover stale leases, pending decisions, uncertain operations, secret exposure, budget overrun and failed environment termination.

PROJECT SPECIFICATION / 26

Revised stack and deployment strategy

Area	Reuse-first proposal
Resource foundation	Evaluate OpenClaw Enterprise/OCC at a pinned revision. Reuse its API, worker, PostgreSQL state, IAM, audit and supported Drivers; validate deployment and upgrade burden.
Product backend	Small TypeScript service for tasks, attempts, attention projection, budget and result acceptance. Dedicated owned schema; call OCC/runtime public APIs.
Runtime/session path	OpenClaw Gateway plus a supported embedded or dedicated Harness. Keep native session authority and permission callbacks; normalize only at product boundary.
Compute path A	OCC Kubernetes Compute for its qualified topology. Kubernetes and node placement are real prerequisites, not hidden free infrastructure.
Compute path B	If OCC fails the fit gate: existing runtime + one managed sandbox (e.g. E2B candidate) behind a provider adapter. No direct Firecracker fleet first.
Secrets / identity	Use existing IAM, secret-store and supported credential bindings. Validate task-scope gaps; do not build cryptography or OAuth from scratch.
UI / artifacts	Reuse native runtime views for controlled internal inspection; product task/review UI plus object storage and existing CI verification.
Workflow engine	Use only one owner per domain. OCC owns infrastructure reconciliation. Product waits use SQL/outbox or Temporal, selected after a durability spike; no second infrastructure controller.

Initial repository additions

```
services/product-tasks   Outcomes, attention, budgets, acceptance
packages/product-api     Stable surface contracts
integrations/occ         Public API mapping; no private DB writes
integrations/runtime     Native session/event/approval correlation
apps/task-workspace      First-party product UI
tests/integration-fit    Reuse qualification and failure tests
```

Pin upstream code/images and maintain an upgrade compatibility suite. Favor composition or upstream extensions over a deep fork. If a fork is necessary, record the patch set, license notices, upgrade owner and replacement path. Do not install OCC, OpenShell, E2B, OpenSandbox and Daytona together merely because each has a control-plane API.

PROJECT SPECIFICATION / 27

Threat model and trust boundaries

Threat	Required mitigation / residual issue
Prompt injection from repo/web/tool	Treat retrieved text as untrusted. Enforce permissions outside instructions. Injection can still corrupt results; verification and review remain necessary.
Cross-tenant access	Tenant checks on every API/object/broker call; scoped keys, isolated environments and negative tests. Never trust client-supplied ownership.
Credential theft / exfiltration	Brokered access, narrow short-lived credentials, egress policy and secret redaction. A credential placed in a shell is accessible to that workload.
Approval substitution	Canonical action hash, request version, identity/target/expiry binding and executor validation.
Untrusted worker events	Authenticate producer; validate lease epoch and schema; never accept worker claims as grants.
Dependency / repository execution	Pinned images, restricted mounts and network; scan dependencies; verifier executes untrusted code in isolation too.
Webhook / OAuth abuse	Signatures, callback validation, replay defense, URL validation and provider allowlists. Guard metadata discovery against SSRF.
Browser session leakage	Separate trust-scoped profiles, controlled remote viewer, secure takeover and no accidental session sharing.
Resource exhaustion	Budgets, quotas, concurrency/depth limits, timeouts, bounded event payloads and admission control.
Unauthorized operator access	Least-privilege administration, auditable break-glass access, no unrestricted secret viewer.

Release-blocking controls

Before public multi-tenant use: pass isolation tests, validate provider security, protect decision endpoints, verify secret handling, test backup restore and establish incident response. The prototype can use synthetic data and low-authority connections until those gates pass. An architectural drawing is not evidence that a sandbox is secure.

Data boundaries

Classify artifacts and outputs. Filter recipient access before handoff or download. Track provider processing/retention terms and deployment regions before onboarding production data. A task deletion must address database records, objects, checkpoints, credentials/grants and backups according to an explicit retention policy.

PROJECT SPECIFICATION / 28

Acceptance and conformance tests

Test	Pass criterion
A01 Task completion	Real runtime produces a revision-bound patch and required evidence; task awaits/records acceptance.
A02 Question round trip	Typed answer reaches exact native request; no unrelated task/session receives it.
A03 Approval enforcement	Denied operation does not execute; changed target/input invalidates prior approval.
A04 Restart while waiting	Backend restart preserves request and decision; delivery/consumption is reconciled.
A05 Duplicate input	Repeated task submission/event/decision creates one logical effect and no duplicate active attempt.
A06 Worker loss	Stale worker loses authority; new owner does not blindly repeat uncertain external writes.
A07 Runtime crash	Exact resume or explicit new attempt shown according to tested capability.
A08 Cancellation	No new authorized work after stop; process/provider termination and cleanup are confirmed or explicitly uncertain.
A09 Tenant isolation	Other tenant cannot read task, artifacts, credentials, events or respond to attention.
A10 Credential revocation	New calls fail after revoke; cached grants are invalidated; in-flight effects are reconciled.
A11 Artifact integrity	Verification references exact result revision and immutable evidence; stale tests do not accept new code.
A12 Adapter substitution	Second integration preserves task/attention contract without editing core task semantics.
A13 Budget and cleanup	Ceiling stops new dispatch; orphan sweeper finds and terminates leaked test resources.
A14 Security bypass	Unmediated shell/browser path cannot acquire forbidden authority in supported task mode.

Test layers

Unit-test state transitions and authorization; contract-test schemas and adapter semantics; integration-test database/outbox/worker behavior; fault-inject crash points around dispatch and decisions; run end-to-end tasks against the pinned real runtime. Use a synthetic external service with non-idempotent writes to expose duplicate-effect failures.

Do not approve release based only on mocked runtime success. Every advertised capability has a versioned fixture and observed result. Provider failure tests include a delayed stop acknowledgment and a provision response lost after the machine was created.

PROJECT SPECIFICATION / 29

Revised phased build plan

Phase	Work	Exit gate
0. Reuse qualification	Run OCC fit spike; native runtime prompts and cancellation; sandbox alternative evaluation; licensing/auth review.	Record PASS/PARTIAL/FAIL per capability; choose composition A or B. No new deployment controller beforehand.
1. Product vertical slice	Reuse chosen resource/runtime foundation; implement task API, correlation and one web task workspace.	Real request produces revision-bound result; native session mapped; no duplicate resource ownership.
2. Attention and recovery gaps	Integrate native permission/question mechanisms; persist product requests; test restarts and expired native authority.	Answer/deny/cancel round trips pass; no stale approval revival; uncertain effects surfaced.
3. Trust and verification	Extend only missing task grants; reuse secret infrastructure, logs and CI; budgets, artifact manifest and review.	Isolation, revocation, evidence and resource-cleanup tests pass.
4. Portability pilot	Add second supported runtime or surface; test the public contract without changing task semantics.	Integration-fit tests pass; costs and operational burden measured.
5. Coordinated work	Reuse native delegation/routines; add child-task ownership, bounded budgets and evidence joins.	No duplicate scheduler or native completion messages; parent cancellation reconciled.
6. Expansion	SDK/API product, private workers or browser use only when requested by validated use cases.	New capability and security gate for each product scope.

Stop conditions

If OCC cannot deploy the selected runtime with an acceptable authentication/isolation path, do not keep adding experimental layers to force it. Choose the lighter runtime/provider route and record why. If neither route passes enforceable approval and cancellation tests, restrict the prototype to low-authority workloads while resolving the missing execution boundary.

What changed from v1.0

Infrastructure controller, IAM, audit, harness and sandbox lifecycle are now reuse candidates. Phase 0 is an adoption decision rather than a greenfield architecture exercise. Product-specific task/attention/result requirements and acceptance tests remain; their implementation owner depends on observed upstream support.

PROJECT SPECIFICATION / 30

Revised backlog: work we actually own

Package	Output	Sourcing / gate
WP01 Reuse qualification	Pinned capability report, license inventory, runtime/provider/auth prototype and decision.	Evaluate OCC before building its equivalents.
WP02 Product contract	Task/attempt/attention/result schemas; OpenAPI; tenant identity mapping.	BUILD product semantics; reuse platform authentication.
WP03 OCC integration	Namespace/Agent/revision mapping, scoped API access, readiness and lifecycle observations.	INTEGRATE; no new Compute controller or private-table writes.
WP04 Runtime correlation	Map native session/turn/approval IDs; stream replay/gap handling; follow-up and cancel.	INTEGRATE existing Gateway/harness; build missing normalization only.
WP05 Attention product	Durable inbox, authorized response, action/request version, delivery and consumption status.	BUILD projection; native executor retains final authority.
WP06 Credential gaps	Task-use grants, missing app connections and revocation reconciliation.	REUSE supported Secrets/credential paths; no bespoke vault.
WP07 Results and evidence	Artifact manifest, revision-bound CI proof, accept/request-changes.	BUILD product review; reuse native diff and CI.
WP08 User experience	Task creation/workspace, attention, connections and result review.	BUILD task UI; reuse channels/SDKs where qualified.
WP09 Durability gaps	Product outbox/timers, stale native requests, uncertain effects and restart recovery.	BUILD only missing task domain behavior; OCC owns deployment retries.
WP10 Operations and cost	Product budgets, telemetry correlation, retention and cleanup verification.	REUSE exporters/provider metrics; extend measured task usage.
WP11 Portability	Second supported integration and executable compatibility report.	INTEGRATE; close vendor-specific semantic gaps explicitly.
WP12 Delegation/routines	Child task/result mapping and bounded schedules.	EXTEND native mechanisms after pilot reliability.

Each ticket includes upstream component/version, tested feature, remaining gap, owner, failure behavior and acceptance test. Delete a build ticket when the upstream path passes the requirement unchanged. Do not estimate saved engineering percentages before the qualification tests.

PROJECT SPECIFICATION / 31

Architecture decisions and adoption risks

Decision	Proposed direction	Closure evidence
Foundation	OCC first evaluation; not unconditional adoption.	Real namespace/Agent/runtime deployment, scoped IAM, replacement/cancel tests and operator burden.
Task vs resource state	Product owns outcomes; OCC owns resources.	Entity mapping and restart test; no dual controller for same Agent/environment.
Runtime	Reuse OpenClaw/qualified Codex path first.	Pinned permission/question/cancel behavior and account authentication.
Sandbox	One supported stack; managed alternative if necessary.	Exact runtime/storage/identity/egress compatibility, cleanup and costs.
OpenShell pairing	Experimental candidate; not initial production assumption.	Matched OCC/OpenShell versions and documented upstream blockers resolved/tested.
Durability	Reuse resource queue; implement only missing product waits.	Failure injection proves decisions and task state survive without repeating effects.
Credentials	Native secret bindings plus scoped product grants.	Model/app/transport credentials distinguished; revocation and isolation tested.
Portability	Public product contract stays independent.	Second integration works; unsupported capabilities are explicit.
Licensing / upgrades	Pin components and retain notices; prefer composition.	SBOM, licenses/terms review, upgrade fixture and patch owner.
Cost / schedule	Measure before selecting self-host vs managed.	Operator time, compute/model usage and missing-feature backlog from spike.

Decision records

Revise ADR-001 to composition over an evaluated foundation; ADR-002 to separate product task state from OCC resource state; ADR-003 to explicit trust scope; ADR-004 to pinned runtime/protocol; ADR-005 to native approval authority plus product projection. Add ADR-009 adoption vs fork and ADR-010 one lifecycle owner per resource.

Build authorization rule. A missing capability demonstrated by a current test is a reason to extend/build. A capability merely absent from a marketing page is not. Source inspection and existing tests are evidence of implementation, not proof that our deployment passes.

PROJECT SPECIFICATION / 32

Worked example: task and events

Task submission - proposed API payload

```
POST /v1/tasks   Idempotency-Key: example-login-fix-01
{
  "project_id": "proj_example",
  "profile_version": "engineering@3",
  "objective": "Fix the login error on an expired session",
  "acceptance_criteria": [
    "Expired session redirects to sign-in",
    "Regression test reproduces and verifies the fix",
    "Existing targeted test suite passes"
  ],
  "runtime_constraints": {"permission_callbacks": true},
  "budget": {"max_duration_seconds": 1800},
  "external_effects": "no_publish"
}
```

Expected durable sequence

Sequence	Observation / transition
1-4	task.submitted -> attempt.allocated -> environment.ready -> runtime.started.
5-7	turn.started -> progress/tool events -> attention.requested: intended redirect behavior.
8-10	decision.recorded -> decision.delivered -> decision.consumed; work resumes.
11-14	artifact.created -> turn.finished -> verification.finished -> ready_for_review.
15-17	result.accepted -> task.succeeded -> cleanup.finished.

Sequence values are illustrative, not a requirement that every task emit exactly these counts. Tool actions and usage may add events. The accepted patch revision and test revision must match. A browser or shell attempt to publish conflicts with no_publish and must be blocked at an enforceable boundary.

Crash between decision and delivery

If the backend stops after event 8, the committed outbox still contains DeliverDecision. On restart, it dispatches the same decision ID to the same request. If the adapter already consumed it, deduplication returns that fact. If the runtime is gone, reconcile the old attempt and supersede the request rather than pretending work resumed.

PROJECT SPECIFICATION / 33

Worked example: approval and adapter manifest

Action approval - required binding

```
{
  "request_id": "req_example", "request_version": 2,
  "type": "action_approval", "task_id": "task_example",
  "attempt_id": "attempt_example", "native_request_ref": "opaque",
  "operation": "repository.create_draft_pr",
  "target": {"connection_id": "conn_example", "repo": "org/repo"},
  "inputs": {"base": "main", "head_revision": "exact-revision"},
  "action_hash": "digest-of-canonical-operation-target-inputs",
  "policy_version": "policy@5", "expires_at": "RFC3339",
  "authorized_responders": ["principal_example"]
}
```

The UI response supplies request_version and approve_once or deny; the backend derives principal identity from authentication. The executor checks the recorded grant, current policy, input hash and lease. A different head revision requires a new action evaluation and approval.

Runtime capability manifest - illustrative

```
{
  "adapter": "example@1.0", "runtime": "pinned-version",
  "protocol": "negotiated-and-pinned",
  "capabilities": {
    "structured_events": true, "permission_callbacks": true,
    "follow_up": "queued", "cancel": "native_then_process_stop",
    "recovery": "restart_from_artifacts", "usage": "partial",
    "concurrent_turns": false, "secure_auth_handoff": false
  },
  "conformance_report_ref": "report-with-observed-test-results"
}
```

Contract publication checklist

Publish OpenAPI for public endpoints; JSON Schemas for events, commands and attention payloads; adapter/provider interfaces; state-transition rules; idempotency and ordering semantics; authentication/authorization requirements; compatibility tests and deprecation policy. Examples above are design illustrations, not executable credentials or verified third-party schemas.

PROJECT SPECIFICATION / 34

Research references and interpretation

Primary sources reviewed on 12 October 2026. URLs identify authoritative documentation; protocol versions are explicit where used. Recheck capabilities and terms when pinning an implementation. The specification's architecture and defaults are proposals, not statements that these sources prescribe this project.

[R1] Grok Bot: overview

https://docs.x.ai/grok-bot/overview

Persistent named Bots, context and shared cloud computer.

[R2] Grok Bot: teams and enterprises

https://docs.x.ai/grok-bot/teams-and-enterprises

Published servers/model/computer architecture, per-user Firecracker microVM and connector-token handling.

[R3] Grok Bot: engineering guide

https://x.ai/bot/guides/grok-bot-for-engineering

Bot harness reference and supervision of Cursor Cloud Agents.

[R4] Grok Bot: collaboration and identity

https://docs.x.ai/grok-bot/chat-and-collaboration

Asynchronous handoffs and user interaction; identity companion: https://docs.x.ai/grok-bot/identity-and-access

[R5] Agent Client Protocol: v1 overview

https://agentclientprotocol.com/protocol/v1/overview

Initialization, session creation/load capability, updates, permissions and cancellation. Pin negotiated version; do not mix v1/v2 semantics.

[R6] Model Context Protocol: authorization, 2025-11-25

https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization

HTTP authorization and distinct STDIO credential path; not a task orchestration protocol.

[R7] Temporal: platform overview

https://docs.temporal.io/temporal

Durable workflow execution and worker/service separation. It does not make arbitrary external side effects exactly-once.

[R8] RFC 9700: OAuth 2.0 Security Best Current Practice

https://www.rfc-editor.org/rfc/rfc9700

OAuth security reference for authorization and credential handling.

[R9] Firecracker: design documentation

https://github.com/firecracker-microvm/firecracker/blob/main/docs/design.md

MicroVM/VMM architecture and isolation model; not a complete environment fleet manager.

[R10] Grok Build: overview

https://docs.x.ai/build/overview

Documented coding agent, headless use and ACP integration; candidate adapter, not verified as Grok Bot's underlying runtime.

Unverified details remain explicit: Grok Bot's internal harness dependencies, workflow engine and message broker are unknown. Other coding runtimes' exact integration capabilities require fresh official-source review and conformance testing before being advertised.

PROJECT SPECIFICATION / 35

Reuse decision: the architecture changes

Recommendation. Evaluate OpenClaw Enterprise as the resource control-plane foundation before building a replacement. Keep a small outcome-oriented product task layer above it. Use an existing runtime and one qualified execution provider. Do not start by assembling a direct Firecracker fleet.

What the research establishes

OCE/OCC documents agent resource management and has a source tree containing a PostgreSQL work queue, controller worker, IAM and audit packages [R11-R14]. OpenClaw supplies runtime/session/channel mechanisms [R24-R27]. Sandbox platforms provide lifecycle/execution layers above a VMM [R18-R21]. These are concrete reuse candidates, not evidence of an already completed end-user product.

What remains our responsibility

The proposed product still needs a durable requested outcome, acceptance criteria, task-to-native-run mapping, cross-surface attention view, result evidence and budget boundaries. A deployed Agent can serve many sessions; an AgentRevision is a deployment snapshot, not a code-change result. Reusing resource lifecycle does not erase this semantic difference.

Decision	Version 1.1 position
Build an entire control plane?	No default greenfield build. Qualify OCC and extend only demonstrated gaps.
Use Firecracker directly?	Not for the first product. It is a VMM requiring surrounding operations [R18].
Reuse a harness?	Yes. Existing OpenClaw/qualified Codex paths before a custom reasoning loop.
Install every candidate?	No. Select one resource owner and one sandbox lifecycle stack per topology.
Preserve portability?	Yes, in our public task/attention contracts and narrow integration boundary.
Confidence	High: component boundaries. Moderate: OCC adoption fit. Unknown: live deployment success until tested.

Research method and limits

Reviewed official product/protocol documentation and inspected OCE source at commit 07590c64e27ac61c9ad60d9ab61424b70281011a. Looked at current feature references and implementation/test paths, not only README claims. No stack was deployed, no model credentials used and no upstream tests executed. Source presence is not a production certification. Rolling main and cached pages can differ; pin the actual revision used.

PROJECT SPECIFICATION / 36

OpenClaw Enterprise: reuse and gaps

Capability	Observed evidence	Fit decision
Resource management	Namespaces, Agents and immutable revisions; API/console and worker [R11-R12].	Reuse for resource lifecycle.
Durable infrastructure work	SQL queue/claims, retry/recovery code and controller worker [R13-R14].	Reuse; do not build another deployment queue.
IAM / audit	Separate packages and exact-resource checks; sanitized audit [R11-R12].	Reuse for OCC resources; extend task authorization.
Harness topology	Embedded OpenClaw; dedicated Codex on Kubernetes [R15].	Qualify exact supported path.
Compute support	Kubernetes deploys; SSH embedded path; Compose preview does not prove deployability [R16].	OCE operations require deliberate topology choice.
Product task outcomes	Reviewed model is deployment/resource-focused; our task acceptance contract is not established.	Build or locate/test an upstream extension.
Universal runtime support	Driver interfaces exist; current Harness combinations are restricted.	Do not advertise arbitrary CLI compatibility.
Credential-free execution	Specific pairing exists with experimental boundaries; general mediation not a proven default.	Treat as qualification gate, not solved security.
Budgets / attention portability	No reviewed feature establishes our complete task budget/inbox contract.	Extend only missing requirements.

Resource state is not task state

Map ProductTenant to an admitted OCC Namespace; ProductProfile to admitted Configuration/Agent references; Attempt to exact AgentRevision plus native session/turn. Keep ResultRevision separate. OCC readiness means a runtime is available, not that the requested software change is correct. A stopped task may leave a reusable Agent service running under an explicit lifecycle policy.

Adoption boundary

Use supported APIs and Driver extension points; avoid direct writes to OCC state. Preserve the original actor and exact-resource checks. A broad backend service key plus self-invented user identity would bypass authorization rather than integrate it. Prefer composition; fork only for a documented gap and maintain a small patch set.

PROJECT SPECIFICATION / 37

OpenClaw: runtime and interaction reuse

Separate OpenClaw from OpenClaw Enterprise

OpenClaw is the agent runtime/gateway ecosystem; OCE/OCC manages deployment resources around admitted runtimes. The Gateway has typed request/response and push events, sessions and channel integrations [R24]. Reuse these capabilities rather than writing a new reasoning loop or every chat transport.

Native capability	What to reuse	What still needs product work
Session / run operations	Existing Gateway protocol and native session controls.	Map task/attempt/native IDs; reconcile event gaps and stop reasons.
Exec approvals	Native requested/resolved events and bound execution plans [R25].	Task inbox projection, responder mapping and stale-request recovery.
Codex Harness bridge	Existing plugin/app-server route and native state [R26].	Version/capability tests; preserve native authority and constraints.
Agent communication	Session tools and native delegation [R27].	Outcome ownership, dependencies, artifact authorization and aggregate budgets.
Channels / UI	Existing channels and Control UI where fit.	First-party task/result UX; scoped end-user admission.
Memory / automations	Existing native mechanisms when verified.	Task templates, policy/budget mapping and retention requirements.

Approval integration rule

Observe the native approval request, persist its product correlation and present it through an authorized surface. Resolve it using the native supported method. The native execution owner decides whether the original run is still eligible. The product must show superseded/expired if native authority is gone; a stored answer cannot resurrect it.

Do not confuse admin UI with user-task API

An operator UI may expose broad runtime state or commands. OCC documents an opt-in native-admin pilot with boundaries different from per-command authorization. Do not embed that privileged view as the default user task product [R12]. Test scoped access through the selected supported ingress/API route.

Assessment: high confidence that the documented mechanisms are reuse candidates; moderate in their exact fit to our persistent attention/recovery contract until tested together.

PROJECT SPECIFICATION / 38

Firecracker and sandbox platform choices

Option	Provides	Still outside its scope
Firecracker VMM	KVM microVM primitive, REST control, device/resource controls and jailer; Apache-2.0 [R18].	Host fleet, image pipeline, workspace API, artifact service, task state, prompts and credential policy.
E2B cloud/runtime	Higher-level sandbox API and runtime/orchestrator; Firecracker-based execution; pause/resume [R19-R20].	Product task/approval semantics; provider/auth integration and operational qualification.
OpenSandbox	Lifecycle and exec APIs, Docker/Kubernetes paths, egress and optional credential vault [R21-R22].	User-task orchestration, verified deployment isolation and app identity grants.
Daytona	API, scheduling, runner/daemon, toolbox, routing and snapshot/volume mechanisms [R23].	Our task/attention/result model; validate current hosting/license path separately.
OCC Kubernetes Compute	Agent runtime and revision provisioning in its supported topology [R15-R16].	Hardware VM isolation is not implied by separate Pods or namespaces.

Select at the highest suitable abstraction

If the requirement is an agent workspace that can run commands and return files, integrate a sandbox API. Choose raw Firecracker only when controlling the VMM/fleet is itself a validated requirement and the team can operate KVM hosts, kernels/images, networking, snapshots, storage and patching. Fast VM launch numbers are not end-to-end agent startup measurements.

Two valid starting routes

OCC route: use its qualified Compute/Harness topology first; a new sandbox provider requires an actual compatible Driver, not an assumed SDK connection. Lightweight route: existing runtime inside one managed sandbox plus the product task layer. E2B is a candidate for this route; select after egress, artifact, runtime transport and pricing tests.

Pause is not business recovery

E2B documents preserved memory/filesystem on pause and reconnection requirements [R20]. That helps runtime continuity, but external sockets, expired credentials and uncertain side effects still need reconciliation. A filesystem-only restore is a new process path; do not promise native session resume from a disk snapshot alone.

PROJECT SPECIFICATION / 39

OpenShell and credential mediation

What OpenShell can replace

OpenShell documents a trusted supervisor outside the untrusted workload, mediated network access, policy controls and approved-endpoint credential injection. Its gateway owns sandbox policy/lifecycle; compute drivers construct the boundary [R17]. It is a candidate to replace custom sandbox enforcement and credential-proxy machinery, not the product task lifecycle.

Important integration limit

OCE's OpenShell Driver uses a specifically documented experimental development path and provider APIs. Its selected release/configuration has qualifications around identity, mounts, routing and production acceptance [R15-R16]. Newer OpenShell documentation does not prove that the OCE pairing supports those newer behaviors. Match and test both revisions before choosing it.

Credential category	Reuse approach / unresolved work
Model API key	Use supported native Secret binding, or a qualified mediation path. An env-mounted key is visible to the executing workload.
App OAuth / API credentials	Use existing connector/secret infrastructure. Product owns which task may use which connection; mediation cannot invent provider scopes.
Runtime transport identity	Separate from model/app credentials; pin protocol and enforce allowed clients. OCE documents transport limitations [R15].
Sandbox provider identity	Provider/gateway supervises workload identity and lifecycle; validate tenant/attempt bindings.
OpenSandbox Vault alternative	Documented egress injection may cover proxy-compatible secrets [R22]; it is not universal OAuth, SSH or browser-session management.

Do not build two brokers

Choose one authority for secret storage, one responsible executor for credential injection and one product grant layer. A product grant references a connection; the native broker performs the call/injection. Avoid copying long-lived credentials into every adapter. Test TLS/protocol compatibility and deny unmediated routes rather than trusting a placeholder convention.

Recommendation: reuse supported credentials for a low-authority prototype. Qualify mediation separately before promising secret-free production workloads. OpenShell is a strong architectural candidate, but the OCE pairing is not a plug-and-play guarantee.

PROJECT SPECIFICATION / 40

Composition options and lifecycle ownership

Layer	Option A: evaluate OCC foundation	Option B: lighter fallback
User product	Our task/attention/result web workspace.	Same public task product and contracts.
Product task state	Our outcome/attempt/evidence state and missing wait semantics.	Same, with direct runtime/provider integration.
Resource control	OCC API/IAM/worker/revisions through supported interfaces.	One selected sandbox provider owns compute lifecycle.
Session execution	Admitted OpenClaw Gateway/Harness; supported Codex path where qualified.	Existing structured runtime in a sandbox; adapter bridges events and decisions.
Environment	OCC Compute plus only a qualified Sandbox combination.	One managed E2B/Daytona candidate, or qualified self-host OpenSandbox.
Secrets / policy	Native supported bindings; selected mediation if qualified.	Provider/native broker plus managed secret store and product grants.
New code	Product semantics, mapping, fit tests and demonstrated gaps.	Product semantics and direct provider/runtime adapters.

Single owner for each kind of state

The product owns task outcome and user attention. OCC owns desired Agent resources and revisions. The runtime owns native session/turn and action authority. The sandbox provider owns compute lifecycle. A second service can observe or request a transition, but cannot concurrently reconcile the same resource as another owner.

Integration flow for option A

Submit product task -> authorize/map to OCC Namespace and admitted Agent -> observe exact runtime readiness -> create a task-owned native session/turn -> map native prompt/event IDs -> project attention -> resolve through native authority -> collect result and verify -> accept outcome -> request appropriate lifecycle cleanup. Agent deployment may be reused, but task workspace/session isolation must be explicit.

Selection rule

Choose A if its supported topology meets trust and product requirements at acceptable operating cost. Choose B if Kubernetes/platform overhead or native constraints outweigh saved resource-management code. Do not combine both owners on the same attempt, or install a second fleet merely to keep every candidate in the diagram.

PROJECT SPECIFICATION / 41

Build-versus-integrate decision matrix

Requirement	Default decision	Our residual work
VMM / sandbox execution	REUSE Firecracker through a platform, or a qualified provider.	Provider normalization and fit tests; no VMM implementation.
Agent deployment controller	EVALUATE OCC REUSE.	Public resource mapping; missing lifecycle adapter only if proven.
Identity / secret storage	REUSE existing platform/managed mechanisms.	Task grant and responder identity mapping; provider connection UX.
Agent reasoning loop	REUSE existing harness.	Native event/permission normalization; unsupported feature disclosure.
Channel transport	REUSE OpenClaw channel/plugin path where supported.	Verified user mapping and task-focused interaction.
Task lifecycle	BUILD outcome-oriented domain.	Objective, criteria, ownership, attempts and review states.
Attention recovery	INTEGRATE native authority; EXTEND missing durable product semantics.	Request version, routing, delivery status, restart reconciliation.
Agent communication	REUSE native tools; EXTEND product contracts.	Parent/child task ownership, data grants and budgets.
Verification / artifacts	REUSE native outputs and CI.	Revision-bound manifest, evidence policy and acceptance.
Scheduling / orchestration	REUSE native routines or one workflow engine where qualified.	Template and authorization mapping; no competing timer/controller.
Usage / observability	REUSE provider metrics and telemetry exporters.	Task ledger, budget reservations and cost reconciliation.
End-user experience	BUILD task workspace.	Task list, status, attention and results over stable backend contracts.

Do not build yet: custom VM orchestration, custom cryptographic vault, new agent harness, bespoke chat transports, duplicate native memory store, generic swarm engine or extra workflow platform without a tested gap. These are deferred work, not hidden prerequisites for the first product.

Proven gap rule: document requirement -> upstream feature/version -> observed result -> remaining mismatch -> smallest extension -> acceptance test. If configuration resolves the mismatch, remove the custom implementation ticket.

PROJECT SPECIFICATION / 42

Qualification experiments before adoption

Experiment	Evidence required	Decision consequence
Q1 OCC real deployment	Pinned setup; Namespace/Agent/revision readiness; bounded real model turn using supported auth.	Adopt resource foundation only if path works; Compose UI alone fails this gate.
Q2 Native attention	Question + exec/plugin approval on exact runtime; approve/deny/cancel; valid responder mapping.	Define reusable native features and missing product bridge.
Q3 Restart and expiry	Restart product/backend/runtime while waiting; reconnect with gap handling; stale approval rejected.	No claim of durable continuation without observed behavior.
Q4 Trust boundary	Per-task/session workspace isolation, credential visibility, forbidden egress and cross-tenant negative tests.	Restrict release scope or change topology on failure.
Q5 Lifecycle ownership	Cancel/stop, revision replacement, provider timeout and cleanup; inspect orphan resources.	One reconciler confirmed; missing adapter paths identified.
Q6 Sandbox alternative	Same task on managed/self-host candidate: processes, files, transport, pause/resume and artifacts.	Choose actual provider, not presumed interchangeability.
Q7 Upgrade / license	Pinned images/licenses, API compatibility and declared migration/rollback procedure.	Record adoption cost and maintained patch set.
Q8 Product portability	Second surface or runtime uses same task/attention contract and identity rules.	Proves modularity; no universal-support claim.

Record the result, not just the experiment

Each report records upstream commit/image/protocol, host/cloud prerequisites, configuration, test inputs, trace, outcome, missing behavior and cost. Use PASS, PARTIAL, FAIL or NOT RUN. Existing upstream test names are not observed passes. No experiments above were executed during this document research.

Cost and maturity assessment

Measure operator setup/upgrade time, required host/cluster services, compute while waiting, storage, provider charges and task usage. MIT/Apache source availability does not imply free hosting, support or commercial provider use. Assess supported releases, API changes, security process and rollback behavior; avoid maturity scores based on repository stars.

PROJECT SPECIFICATION / 43

Adoption operations, licensing and fallback

Licensing inventory from reviewed sources

Component	Reviewed source license / hosting boundary
OpenClaw Enterprise	MIT repository [R11]; third-party components retain their licenses. Pin source and preserve notices.
OpenClaw	Official repository documents MIT; plugins/providers may have separate terms [R24].
Firecracker	Apache-2.0, with stated third-party code notices [R18]. Requires owned hosting if used directly.
E2B Runtime	Apache-2.0 repository [R19]; managed service plans and limits are separate.
OpenSandbox	Apache-2.0 repository [R21]; deployed runtime/backend and images have their own requirements.
OpenShell	Apache-2.0 repository [R17]; integration version and production qualification remain separate.
Daytona	Use current hosted architecture as a candidate [R23]. This review does not establish a complete current self-host source/license path; verify before selecting.

Avoid a permanent deep fork

First integrate public APIs. If an extension is needed, use the declared Driver/plugin interface or propose upstream changes. A local fork must have a tracked patch inventory, upgrade owner, compatibility fixtures and sunset criteria. Do not import private queue/IAM packages as stable SDKs unless upstream explicitly supports that contract.

Fallback that preserves the project

If OCC fails Q1-Q5 or is operationally too heavy, retain the product task/attention/result API and swap its resource integration for an existing runtime plus one sandbox service. Implement only task coordination gaps. A failed adoption test is not a reason to create a Firecracker fleet, credential proxy and harness simultaneously.

Specification authority after this revision

Sections 9-25 remain required product behavior and acceptance boundaries. Sections 8, 26, 29-31 and 35-43 define sourcing and now supersede greenfield assumptions from v1.0. The next deliverable is a qualification report with a selected topology and a reduced implementation backlog, not a claim that these components already interoperate.

PROJECT SPECIFICATION / 44

Reuse research references: platform and runtime

Reviewed 12 October 2026. OCE source inspection pinned at 07590c64e27ac61c9ad60d9ab61424b70281011a. Feature references take precedence over historical matrices. Documentation/source evidence is distinguished from live verification.

[R11] OCE README / license / code layout

Open authoritative source

API/console, contracts, lifecycle, IAM and audit organization; MIT license.

[R12] OCE architecture

Open authoritative source

Implemented system versus remaining work; resource ownership and native-admin limits.

[R13] OCE PostgreSQL controller work queue

Open authoritative source

Inspected source: durable claims, lease state, SKIP LOCKED and recovery; no tests executed.

[R14] OCE controller worker

Open authoritative source

Inspected implementation and integration-test paths; infrastructure reconciliation, not product task acceptance.

[R15] OCE Harness execution / security

Open authoritative source

Reviewed supported topologies and authentication. Companion: docs/reference/security/runtime-isolation.md.

[R16] OCE Compute and OpenShell qualification

Open authoritative source

Current preamble versus historical matrix; companion: docs/reference/drivers/openshell-sandbox.md.

[R24] OpenClaw Gateway architecture

Open authoritative source

Native Gateway/session/channel interface; repository license: github.com/openclaw/openclaw.

[R25] OpenClaw exec approvals

Open authoritative source

Native request/resolve flow, execution binding and invalidation.

[R26] OpenClaw Codex Harness

Open authoritative source

Existing native bridge, supported requirements and execution ownership.

[R27] OpenClaw session tools / restart

Open authoritative source

Native communication; companion: docs.openclaw.ai/gateway/restart-recovery.

PROJECT SPECIFICATION / 45

Reuse research references: execution and evidence

[R17] NVIDIA OpenShell architecture and license

https://docs.nvidia.com/openshell/latest/about/architecture

Latest page identified v0.1.3 during review; gateway/supervisor/isolation architecture. License: github.com/NVIDIA/OpenShell.

[R18] Firecracker official site

https://firecracker-microvm.github.io/

VMM, KVM, REST control and jailer. This review makes no managed-service availability or performance guarantee.

[R19] E2B Runtime repository / architecture

https://github.com/e2b-dev/runtime

Official infra URL redirected to runtime during review; Apache-2.0 backend and component scope. Architecture: docs/ARCHITECTURE.md.

[R20] E2B sandbox persistence

https://docs.e2b.dev/sandbox/persistence

Pause/resume memory/filesystem modes and client reconnection caveats.

[R21] OpenSandbox official repository

https://github.com/opensandbox-group/OpenSandbox

Apache-2.0 and lifecycle/execution/runtime integration scope.

[R22] OpenSandbox architecture / egress

https://github.com/opensandbox-group/OpenSandbox/blob/main/docs/architecture/index.md

Lifecycle control plane, execution components and optional credential-vault injection; validate selected runtime.

[R23] Daytona architecture

https://www.daytona.io/docs/en/architecture/

API, manager, runners, toolbox, proxy and snapshot/volume boundaries; current self-host licensing not established in this review.

Additional inspected OCE files at the pinned commit

docs/reference/agent-plugins.md; docs/reference/drivers/plugin-bundled.md; docs/reference/drivers/channel.md; docs/reference/credential-sources.md; docs/reference/security/runtime-isolation.md. These narrow claims about generic plugin approvals, channel lookup and credential visibility. Existing test files include postgres-worker-stale-claim, sandbox-driver-openshell-k3d-real and plugin-driver-real; their presence is not a pass result.

Evidence and confidence rules

Documented = official reference says the scoped feature exists. Source-inspected = relevant implementation is present at the pinned revision. Tested = executed in our selected deployment with recorded evidence. Only the first two apply to this research. Component selection recommendations are architectural judgments with moderate confidence until the qualification experiments pass.